NAMA :REYHAN GILANG RAMADHAN

KELAS :SI-IIIA

# JAWABAN UTS

## JAWABAN A1

### 1. Class

Pengertian: Class adalah cetak biru atau rancangan untuk membuat objek. Class menentukan atribut dan method yang dimiliki oleh objek.

Contoh kode:

```
class Mahasiswa:    nama = "Andi"
```

### 2. Object

Pengertian: Object adalah hasil atau instance dari sebuah class yang memiliki atribut dan dapat menjalankan method.

Contoh kode:

```
class Mahasiswa:    nama = "Andi"mhs1 =
Mahasiswa()print(mhs1.nama)
```

Output:

```
Andi
```

### 3. Attribute

Pengertian: Attribute adalah variabel yang digunakan untuk menyimpan data atau karakteristik yang dimiliki oleh sebuah class atau objek.

Contoh kode:

```
class Mahasiswa:    def __init__(self, nama):        self.nama
= namamhs1 = Mahasiswa("Budi")print(mhs1.nama)
```

Output:

```
Budi
```

### 4. Method

Pengertian: Method adalah fungsi yang didefinisikan di dalam class dan digunakan untuk menjalankan suatu tindakan atau perilaku objek.

Contoh kode:

```
class Mahasiswa:    def belajar(self):        print("Mahasiswa sedang belajar")mhs1 = Mahasiswa()mhs1.belajar()
```

Output:

```
Mahasiswa sedang belajar
```

**5. Constructor**

Pengertian: Constructor adalah method khusus `__init__()` yang dijalankan secara otomatis ketika objek dibuat. Constructor digunakan untuk memberikan nilai awal pada atribut objek.

Contoh kode:

```
class Mahasiswa:    def __init__(self, nama):        self.nama = namamhs1 = Mahasiswa("Andi")print(mhs1.nama)
```

Output:

```
Andi
```

**JAWABAN A2**

**1. Encapsulation (Enkapsulasi)**

Pengertian: Encapsulation adalah konsep membungkus data dan method dalam satu class serta membatasi akses langsung terhadap data tertentu.

Penggunaan: Melindungi data agar tidak diubah secara langsung dari luar class.

Contoh kode:

```
class Mahasiswa:    def __init__(self, nilai):
self.__nilai = nilai    def get_nilai(self):        return self.__nilaimhs = Mahasiswa(90)print(mhs.get_nilai())
```

Output:

```
90
```

Penjelasan: Attribute `__nilai` dibuat privat menggunakan dua garis bawah. Data tersebut diakses melalui method `get_nilai()`.

**2. Inheritance (Pewarisan)**

Pengertian: Inheritance adalah konsep ketika sebuah class mewarisi atribut dan method dari class lain.

Penggunaan: Menggunakan kembali kode dari class induk tanpa harus menulis ulang seluruh program.

Contoh kode:

```
class Hewan:    def makan(self):        print("Hewan sedang makan")class Kucing(Hewan):    def suara(self):        print("Meong")kucing = Kucing()kucing.makan()kucing.suara()
```

Output:

```
Hewan sedang makan
Meong
```

Penjelasan: Class `Kucing` mewarisi method `makan()` dari class `Hewan`.

**3. Polymorphism (Polimorfisme)**

Pengertian: Polymorphism adalah kemampuan objek yang berbeda untuk menggunakan method dengan nama yang sama tetapi memiliki perilaku yang berbeda.

Penggunaan: Membuat program lebih fleksibel karena satu method dapat memiliki implementasi berbeda.

Contoh kode:

```
class Kucing:    def suara(self):        print("Meong")class Anjing:    def suara(self):        print("Guk guk")for hewan in [Kucing(), Anjing()]:    hewan.suara()
```

Output:

```
Meong
Guk guk
```

Penjelasan: Kedua class memiliki method `suara()`, tetapi menghasilkan perilaku yang berbeda.

**4. Abstraction (Abstraksi)**

Pengertian: Abstraction adalah konsep menyembunyikan detail implementasi dan hanya menampilkan fungsi penting yang diperlukan pengguna.

Penggunaan: Mempermudah penggunaan program tanpa harus mengetahui proses internalnya.

Contoh kode:

```
from abc import ABC, abstractmethodclass Kendaraan(ABC):
@abstractmethod    def berjalan(self):        passclass
Mobil(Kendaraan):    def berjalan(self):        print("Mobil
berjalan")mobil = Mobil()mobil.berjalan()
```

Output:

```
Mobil berjalan
```

Penjelasan: Class `Kendaraan` menentukan method abstrak `berjalan()`. Class `Mobil` harus mengimplementasikan method tersebut agar dapat dibuat menjadi objek.

**JAWABAN A3**

**1. Konsep OOP pada `self.__nama` dan `@property`**

Konsep OOP yang diterapkan adalah Encapsulation (Enkapsulasi).

Alasan teknis:

- `self.__nama` menggunakan dua garis bawah (__) untuk menerapkan name mangling, sehingga atribut tidak dapat diakses secara langsung menggunakan nama `__nama` dari luar class.
- `@property` digunakan untuk menyediakan akses terhadap atribut privat melalui method yang dipanggil seperti atribut biasa.
- Dengan demikian, data dapat dilindungi dan aksesnya dapat dikendalikan melalui interface yang disediakan class.

Contoh:

```
pengguna = Pengguna("Andi",
"andi@email.com")print(pengguna.nama)
```

Output:

```
Andi
```

**2. Konsep OOP pada `class Mahasiswa(Pengguna)` dan fungsi `super()`**

Konsep OOP yang diterapkan adalah Inheritance (Pewarisan).

`class Mahasiswa(Pengguna)` menunjukkan bahwa class `Mahasiswa` merupakan subclass atau class turunan dari class `Pengguna`.

Fungsi `super()` adalah untuk memanggil method dari superclass atau class induk.

Pada kode:

```
super().__init__(nama, email)
```

`super()` memanggil constructor `__init__()` milik class `Pengguna` untuk menginisialisasi atribut nama dan email.

Dengan demikian, class `Mahasiswa` dapat menggunakan kembali proses inisialisasi dari class induk tanpa menulis ulang kode tersebut.

**3. Alasan `self.nama` dapat dipanggil pada method `tampilkan_peran()`**

`self.nama` dapat dipanggil karena class `Pengguna` menyediakan method getter menggunakan `@property` yang bernama `nama`.

Meskipun atribut `self.__nama` bersifat privat, method `nama` dapat mengakses atribut tersebut dari dalam class `Pengguna` dan mengembalikan nilainya.

Class `Mahasiswa` mewarisi property `nama` dari class `Pengguna`, sehingga property tersebut dapat digunakan oleh subclass.

Contoh:

```
def tampilkan_peran(self):    return f"{self.nama} adalah mahasiswa dengan NIM {self.nim}"
```

## Jawaban Bagian B — Coding dan Problem Solving

**Soal B1 — Class dan Encapsulationclass Buku:**

```
lass Buku:
    def __init__(self, judul, penulis):
        self.judul = judul
        self.penulis = penulis
        self.__tersedia = True

    @property
    def tersedia(self):
        return self.__tersedia

    def pinjam(self):
        if self.__tersedia:
            self.__tersedia = False
            return True
        return False

    def kembalikan(self):
        self.__tersedia = True

    def __str__(self):
        status = "Tersedia" if self.__tersedia else "Dipinjam"
        return f"{self.judul} — {self.penulis} [{status}]"
```

```
# Program utama
buku1 = Buku("Laskar Pelangi", "Andrea Hirata")
buku2 = Buku("Bumi", "Tere Liye")

# Meminjam buku pertama
buku1.pinjam()

# Menampilkan status kedua buku
print(buku1)
print(buku2)
```

OUTPUT

```
Laskar Pelangi – Andrea Hirata [Dipinjam]
Bumi – Tere Liye [Tersedia]
```

Penjelasan:

- `__tersedia` merupakan atribut privat untuk melindungi status buku.
- `@property` digunakan untuk membaca status ketersediaan.
- `pinjam()` mengubah status buku menjadi tidak tersedia.
- `kembalikan()` mengubah status buku menjadi tersedia.
- `__str__()` menghasilkan informasi buku dalam format yang ditentukan.

**Soal B2 — Inheritance dan Polymorphism**

```
class Buku:
    def __init__(self, judul, penulis):
        self.judul = judul
        self.penulis = penulis
        self.__tersedia = True

    @property
    def tersedia(self):
        return self.__tersedia

    def pinjam(self):
        if self.__tersedia:
            self.__tersedia = False
            return True
        return False

    def kembalikan(self):
        self.__tersedia = True

    def __str__(self):
        status = "Tersedia" if self.__tersedia else "Dipinjam"
```

```
        return f"{self.judul} - {self.penulis} [{status}]"


class BukuDigital(Buku):
    def __init__(self, judul, penulis, format_file):
        super().__init__(judul, penulis)
        self.format_file = format_file

    def __str__(self):
        status = "Tersedia" if self.tersedia else "Dipinjam"
        return (
            f"{self.judul} - {self.penulis} "
            f"[{status}] ({self.format_file})"
        )

    def unduh(self):
        if self.tersedia:
            return (
                f"Mengunduh {self.judul} "
                f"dalam format {self.format_file}."
            )
        return "Buku tidak tersedia."


def tampilkan_info(daftar_buku):
    for buku in daftar_buku:
        print(buku)


# Program utama
buku1 = Buku("Laskar Pelangi", "Andrea Hirata")
buku2 = BukuDigital("Bumi", "Tere Liye", "PDF")

# Meminjam buku digital
buku2.pinjam()

# Membuat daftar buku
daftar_buku = [buku1, buku2]

# Menampilkan informasi semua buku
tampilkan_info(daftar_buku)

# Menguji fitur unduh
print(buku2.unduh())
```

outputnya

```
Laskar Pelangi - Andrea Hirata [Tersedia]
Bumi - Tere Liye [Dipinjam] (PDF)
Buku tidak tersedia.
```

Penjelasan:

- Inheritance: `BukuDigital` mewarisi atribut dan method dari class `Buku`.
- `super()`: Memanggil constructor parent untuk menginisialisasi judul dan penulis.
- Polymorphism: Method `__str__()` pada `BukuDigital` memiliki implementasi berbeda dari class `Buku`.
- `tampilkan_info()`: Dapat menampilkan kedua jenis objek tanpa pengecekan tipe eksplisit.
- `unduh()`: Memeriksa status ketersediaan sebelum mengembalikan pesan unduhan.

**Soal B3 — Analisis dan Perbaikan Kode**

Identifikasi tiga kesalahan atau kelemahan

1. Attribute saldo dapat diakses langsung dari luar class.
   Attribute `saldo` bersifat publik sehingga dapat diubah tanpa melalui proses validasi.
2. Method `setor()` tidak melakukan validasi jumlah uang.
   Pengguna dapat menyetor nilai nol atau negatif yang menyebabkan saldo tidak sesuai.
3. Method `tarik()` tidak melakukan validasi penarikan.
   Pengguna dapat menarik jumlah negatif atau melebihi saldo sehingga saldo menjadi tidak valid.

Kode yang telah diperbaiki

```
class RekeningBank:
    def __init__(self, pemilik, saldo):
        self.pemilik = pemilik
        self.__saldo = saldo

    @property
    def saldo(self):
        return self.__saldo

    def setor(self, jumlah):
        if jumlah <= 0:
            print("Setoran harus lebih dari nol.")
            return False

        self.__saldo += jumlah
        print(f"Setor berhasil: Rp{jumlah}")
        return True

    def tarik(self, jumlah):
        if jumlah <= 0:
            print("Penarikan harus lebih dari nol.")
            return False

        if jumlah > self.__saldo:
            print("Saldo tidak mencukupi.")
```

```
            return False

        self.__saldo -= jumlah
        print(f"Penarikan berhasil: Rp{jumlah}")
        return True

    def tampilkan_saldo(self):
        print(f"Saldo {self.pemilik}: Rp{self.__saldo}")


# Program utama
rekening = RekeningBank("Andi", 100000)

# Menampilkan saldo awal
rekening.tampilkan_saldo()

# Setor uang
rekening.setor(50000)

# Tarik uang yang valid
rekening.tarik(30000)

# Tarik uang melebihi saldo
rekening.tarik(200000)

# Tarik jumlah negatif
rekening.tarik(-10000)

# Menampilkan saldo akhir
rekening.tampilkan_saldo()
```

output

```
Saldo Andi: Rp100000
Setor berhasil: Rp50000
Penarikan berhasil: Rp30000
Saldo tidak mencukupi.
Penarikan harus lebih dari nol.
Saldo Andi: Rp120000
```

**3. Penjelasan perbaikan**

- `__saldo` digunakan sebagai atribut privat untuk melindungi data saldo.
- `@property` memungkinkan saldo dibaca dari luar class tanpa dapat diubah langsung melalui assignment biasa.
- `setor()` hanya menerima jumlah uang yang positif.
- `tarik()` menolak jumlah negatif, nol, dan penarikan yang melebihi saldo.
- Method mengembalikan `True` jika transaksi berhasil dan `False` jika transaksi ditolak.

Kesimpulan: Penerapan encapsulation dan validasi pada class `RekeningBank` membuat pengelolaan saldo lebih aman serta mencegah transaksi yang tidak sesuai aturan.